AIML THEME

Problem Statement - RAG Forge: Enterprise Retrieval AI with Re-Ranker

Build an advanced Retrieval-Augmented Generation (RAG) system that can ingest large, messy document collections and answer user questions with grounded, cited, and re-ranked results.

The goal is to replace naive “LLM chat over PDFs” with a production-style RAG pipeline that includes chunking, embeddings, vector search, re-ranking, context building, and answer citation.

Target Users

- Knowledge Workers – ask questions over policies, manuals, reports.
- Analysts/Students – extract precise answers with references.
- Admins – upload and manage document collections.

Authentication

- User signs up / logs in.
- OTP-based password reset.
- Redirected to RAG Dashboard.

Dashboard View

- Landing page shows the health and activity of the RAG system.
- Dashboard KPIs
- Total Documents Indexed
- Total Chunks Created
- Embedding Count
- Queries Processed
- Avg Retrieval Time
- Avg Answer Confidence Score
- Dynamic Filters
- By document collection
- By file type (PDF, DOCX, TXT, HTML)
- By embedding model

- By date indexed

Navigation

- Document Collections
- Create/delete collections
- Upload multiple files
- View parsing status
- Chunking & Embeddings
- Show chunk size & overlap
- Generate embeddings
- Store in vector DB
- Retrieval Pipeline
- Vector Search (Top-K)
- Re-Ranker Layer (must)
- Context Builder
- Ask AI (Chat Interface)
- Ask questions
- View retrieved chunks
- See re-ranking scores
- Get cited answers
- Query Logs
- Question
- Retrieved docs
- Final answer
- Latency
- Settings
- Embedding model selection
- Chunk size / overlap
- Top-K value
- Profile Menu
- My Profile
- Logout

Core Features

1) Document Ingestion

- Upload documents (PDF/DOCX/TXT/HTML).

- System must:
- Extract text
- Clean noise
- Split into chunks with overlap

2) Embedding & Vector Storage (You can use ChromaDB)

- Generate embeddings for each chunk
- Store in a vector database
- Show embedding count

3) Retrieval with Re-Ranker (Important)

- When user asks a question pipeline must be:
  Query → Embedding → Vector Search (Top-K) → Re-Ranker model → Best N chunks → Context → LLM Answer
- Re-ranker should improve relevance over raw similarity search.

4) Context Builder

- Combine top re-ranked chunks
- Trim to token limit
- Preserve source references

5) Grounded Answer Generation

- LLM must answer only from provided context
- Show citations (document name + chunk id)
- If answer not found → say “Not found in knowledge base”

6) Query Analytics

- Log for every query:
- Retrieved chunks
- Re-rank scores
- Final context
- Response time

✨ Additional Features (for bonus points)

- Hybrid search (keyword + vector)
- Streaming response
- Upload very large PDFs (100+ pages)
- Compare answer quality with vs without re-ranker
- Confidence score for answer

Simplified Flow Example

Step 1:

Upload 3 policy PDFs (300 pages total)

→ System creates ~2000 chunks → embeddings stored

Step 2:

User asks:

"What is the maternity leave policy duration?"

Step 3:

- Vector DB returns top 15 chunks
- Re-ranker selects best 5
- Context built
- LLM answers with citation:

"Maternity leave is 26 weeks [Policy.pdf | Chunk 184]"